code: https://colab.research.google.com/drive/1dII1Ecra18jJVdT4RPJNtKOqcY3f21hN?usp=sharing

Advanced Information Theory-Based Approach to Optimal Hangman Strategy

Executive Summary

This document presents a detailed analysis of an advanced Hangman strategy that combines information theory, recursive pattern matching, and dynamic dictionary management to achieve superior performance. The approach moves beyond simple letter frequency analysis to leverage entropy-based decision making in a deeply recursive manner, achieving approximately 75% success rate across all games. The strategy employs a novel recursive application of information theory both horizontally (across game steps) and vertically (within each step's decision tree).

1. Theoretical Foundation

1.1 Information Theory Principles

The strategy's foundation rests on Claude Shannon's information theory, specifically the concept of entropy as a measure of information content. The key innovation is applying this recursively at multiple levels:

- Entropy (H) for a letter choice is calculated as: $H = -\Sigma\ p(x) * \log_2(p(x))$ where p(x) is the probability of each possible outcome
- The recursive application occurs in two dimensions:
    1. Horizontal Recursion: Across game steps
    2. Vertical Recursion: Within each step's decision tree

1.2 Recursive Decision Making

The strategy implements a two-level recursive approach:

1. Level 1 (Game Step Recursion):
    - Each game step uses information from previous steps
    - Updates probability distributions based on new information
    - Maintains a dynamic decision tree across turns
2. Level 2 (Decision Tree Recursion):
    - For each potential letter choice, recursively explores all possible word matches
    - Calculates entropy impact through entire remaining word space
    - Considers all possible outcomes down to individual word nodes

2. Implementation Architecture

2.1 Core Data Structures

2.1.1 Optimization Evolution

The implementation underwent significant optimization:

Original Implementation (12 minutes per execution):

```
# List-based tree structure
letter_dict_pos = {}
for char in range(97, 123):
    letter_dict_pos[chr(char)] = {}
```

Optimized Implementation (3 seconds per execution):

```
# Numpy array-based indexing
letter_dict_pos = np.full(((26,45,9)), {}, dtype=object)
letter_dict_pos_index = np.zeros((26,45,9))
letter_dict_pos_char_numbered =
np.copy(letter_dict_pos_index).astype(int)
```

Key Optimizations:

- Replaced nested dictionary trees with numpy arrays
- Implemented indexed access patterns
- Used pre-allocated arrays for position tracking
- Employed vectorized operations for pattern matching

2.1.2 Dynamic Dictionary Management

```
class HangmanAPI:
    def __init__(self):
        self.full_dictionary = cached_words.copy()
        self.current_dictionary = []
        self.letters_absent = []
        self.guessed_letters = []
```

2.2 Recursive Entropy Calculation

2.2.1 Multi-level Recursion

```
def entropy_calc(self, word_dict):
    letter_dict_entropy = {}

    # First level recursion - across all possible letters
    for char_index in range(26):
        # Second level recursion - through all possible word matches
        for word_length in range(45):
            for occurrence_count in range(9):
```

```
                # Calculate entropy contribution at each node
                self.calculate_node_entropy(char_index,
word_length, occurrence_count)
```

2.2.2 Optimization Details

The transformation from list-based to array-based processing yielded dramatic improvements:

1. Memory Access Patterns:
    - Original: O(n) dictionary lookups
    - Optimized: O(1) array indexing
2. Pattern Matching:
    - Original: Recursive tree traversal
    - Optimized: Vectorized operations
3. Space Efficiency:
    - Original: Dynamic allocation
    - Optimized: Pre-allocated arrays

3. Performance Analysis

3.1 Success Rate Metrics

Based on extensive testing:

1. Overall Success Rate: ~75%
2. Performance Breakdown:
    - Average Guesses: 4.2 per game
    - Consistent performance across word lengths
    - High efficiency in early-game letter selection

3.2 Computational Complexity

1. Time Complexity:
    - Entropy Calculation: O(N * L * P)
        - N: dictionary size
        - L: word length
        - P: unique letter positions
    - Pattern Matching: O(N * L)
    - Letter Selection: O(26 * log(26))
2. Space Complexity:
    - Dictionary Storage: O(N * L)
    - Pattern Matrix: O(26 * 45 * 9)
    - Working Set: O(N)

4. Technical Innovations

4.1 Novel Approaches

1. Dual-Recursive Information Theory:
    - Horizontal recursion across game steps
    - Vertical recursion through decision trees
    - Complete exploration of possibility space
2. Dynamic Pattern Matching:
    - Updates pattern constraints efficiently
    - Maintains optimal dictionary subset
    - Handles multiple pattern interpretations

4.2 Information Theoretical Advances

1. Multi-dimensional Information Modeling:
    - Letter frequency information
    - Position information
    - Pattern-based information
    - Combination weighing
2. Adaptive Information Valuation:
    - Recomputes information value based on game state
    - Adjusts for remaining uncertainty
    - Optimizes for minimum guess count

5. Limitations and Future Work

5.1 Current Limitations

1. Computational Constraints:
    - Memory usage with large dictionaries
    - Processing time for long words
    - Edge case handling overhead
2. Strategic Limitations:
    - Dependence on training dictionary
    - Sub-optimal handling of completely unknown patterns
    - Fixed entropy calculation approach

5.2 Cost-Based Recursive Optimization

The current entropy-based approach could be extended to incorporate sophisticated cost functions that consider the structure and characteristics of the word-node tree. This would create a more nuanced decision-making process that accounts for tree complexity and depth.

5.2.1 Tree-Based Cost Functions

1. Node Count Based:

```
def node_count_cost(depth, num_nodes, num_branches):
    # Simple summation
    return depth + num_nodes + num_branches

    # Exponential with depth
    return num_nodes * math.exp(depth)

    # Multiplicative
    return depth * num_nodes * num_branches
```

2. Depth-Weighted Approaches:

```
def depth_weighted_cost(depth, num_nodes, num_branches):
    # Square depth scaling
    return (depth ** 2) * num_nodes

    # Logarithmic node scaling
    return depth * math.log(num_nodes + 1)
```

3. Branch Complexity Integration:

```
def branch_aware_cost(depth, num_nodes, num_branches):
    # Branch path complexity
    return depth * (num_nodes / num_branches)

    # Branch density scaling
    return (depth * num_nodes) * math.log(num_branches + 1)
```

5.2.2 Cost Function Considerations

1. Tree Depth Impact:
    - Linear scaling may undervalue deep paths
    - Exponential scaling emphasizes path length
    - Quadratic scaling provides balanced emphasis
2. Node Distribution Effects:
    - Dense vs sparse tree structures
    - Branch distribution patterns
    - Path redundancy considerations
3. Branch Factor Influence:
    - High branching factors increase search space
    - Low branching suggests more certain paths
    - Optimal branching factor balancing

5.2.3 Implementation Strategy

1. Cost Function Selection:
    - Match cost function to word pattern characteristics

    - Adapt weights based on game progress
    - Consider computational overhead
2. Dynamic Weight Adjustment:
    - Tune weights based on success patterns
    - Adapt to different word lengths
    - Balance exploration vs exploitation

5.3 Additional Potential Improvements

1. Technical Enhancements:
    - Implement progressive dictionary loading
    - Further optimize matrix operations
    - Add parallel processing for entropy calculations
2. Strategic Improvements:
    - Incorporate machine learning for pattern recognition
    - Add adaptive entropy weighting
    - Implement pattern-based prediction

6. Conclusion

This advanced Hangman strategy represents a significant improvement over traditional approaches by leveraging a dual-recursive application of information theory and sophisticated pattern matching. The combination of entropy-based decision making, efficient dictionary management, and optimized implementation results in a highly effective solution achieving a 75% success rate.

The strategy's success demonstrates the power of applying information theory recursively at multiple levels of decision making under uncertainty. The optimization journey from a list-based to array-based implementation showcases the importance of data structure selection and processing efficiency in real-world applications.

Future work could focus on further optimization and machine learning integration, but the current implementation provides a robust and effective solution to the Hangman challenge.